12
Design a Shipping Locker System

In this chapter, we will design a shipping locker system similar to UPS, FedEx, or Amazon Locker. It offers customers a compact and secure way to pick up their packages. The system manages locker availability, assigns incoming packages to appropriate lockers, and ensures a smooth package retrieval process for customers.

To tackle this vision of a convenient package locker system, we explore what it needs to do.

Shipping Locker System

Requirements Gathering

The first step in designing a shipping locker system is to clarify the requirements and narrow down the scope. Here is an example of a typical process at Amazon-level.

Interviewer: Let's design a shipping locker system. Imagine you've been asked to implement a shipping locker over, you're tasked with building the system. What's your first question?

Candidate: Does the system support multiple locker sizes?

Interviewer: Yes, the system has lockers of different sizes. The design should ensure packages are assigned to the smallest available locker that fits, optimizing space use.

Candidate: What happens if a customer's package goes to the locker, and he does not retrieve it in a reasonable time?

Interviewer: Without a pickup time, the system should have an open locker rate that can be adjusted later. If a locker is full and a package is locked inside, the customer gets a notification with the locker's location and a unique access code. They just punch in the code and the locker opens. The system should then mark the locker as available for the next package.

Candidate: Is there a time limit on how long a locker can be reserved?

Interviewer: Yes, lockers should be reserved for a maximum of 24 hours by default. We've got a locker policy that gives customers a free Trial at, and we want to stick with that. We should also be able to change this rule later. We'll need to track how long each locker has been on the system's clock, and if the package sits there past a "Reserved Period," which is also predefined, one of our staff members will need to be notified to manually handle the situation.

Candidate: Should the system support real-time locker availability updates, so customers can see which lockers are available?

Interviewer: For simplicity's sake, let's let an external service take care of the actual payment processing. That's not something the system itself has to worry about.

Requirements

Here are the key functional requirements we identified:

- The system should support multiple locker sizes and assign packages to the smallest available option that fits, optimizing storage efficiency.
- The system should be able to scan each package's barcode with a barcode scanner.
- Customers should be able to scan their assigned locker with a barcode scanner.
- The system should manage locker availability, including tracking which lockers are occupied, reserved, or available.
- Notifications should be sent to scan users when assigned locker with a unique access code.
- The system should enforce a maximum reservation time of 24 hours, after which the locker is marked as available for other packages.
- Notifications should be sent to the system's staff when a package exceeds the reservation time.

Below are the non-functional requirements:

- The system should handle a high volume of locker operations per site without performance degradation, accommodating busy pickup times.
- The system must ensure data integrity, including accurate locker availability and package assignments, to prevent errors and customer dissatisfaction.
- The system should be easy to operate and maintain, with clear interfaces for monitoring locker status and managing exceptions.

Design Class Diagram

Now that we know the core objects and their roles, the next step is to create classes and methods that turn the requirements into a clear blueprint for our shipping locker system.

Locker

The locker class represents a physical storage unit in the shipping locker. It includes the following attributes:

- **LockerSize enum**: Represents the size of the locker.
- **isOccupied**: A boolean indicating whether the locker is currently occupied.
- **lockerLastPickedUpDate**: Tracks the date when the locker was last emptied.
- **String accessCode**: A unique security code required for package retrieval.

Below is the representation of this class:

LockerSize

The LockerSize class represents the predefined sizes of lockers available in the system. Each size is associated with specific dimensions (height, width, and depth) to accommodate different package sizes.

- **String displayName**: A human-readable name for the locker size (e.g., "Small", "Medium", "Large").
- **double getMaxHeight()**: Returns the maximum height for this locker size.
- **double getMaxWidth()**: Returns the maximum width for this locker size.
- **double getMaxDepth()**: Returns the maximum depth for this locker size.

Design choice: We use an enum here because locker sizes, such as Small, Medium, and Large, are a fixed set of options that don't change during runtime, ensuring type safety and simplicity.

Site

The Site class models a physical location containing a collection of lockers, organized by their size.

Key functionalities include:

- **Locker findAvailableLockerForSize(LockerSize size)**: This method searches for an already locker that matches the exact size you need, like "medium" or "large."
- **boolean canAccommodate(Package pkg, Date date)**: This method takes a package, finds a locker that fits, and tells you if it works. It also checks if the package can physically fit and if the locker isn't already full.

The locker's at each site are managed using a map-based structure, where the key is a LockerSize and the value is a set of lockers for that size. This ensures quick access to lockers of a specific size when needed.

ShippingPackage

We have created ShippingPackage as an interface to establish a standard for all package types within the locker system. It defines key methods that any package type must implement to ensure compatibility with locker storage and retrieval processes. The ShippingPackage interface provides a flexible foundation for managing various package types while maintaining consistent behavior across the system.

A StandardPackage class implements a basic set of core details for a package's lifecycle in the locker system, such as PREPARED, STORED, and RETRIEVED. It manages essential attributes like package volume, updates one consistent and maintains a predefined status, enhancing type safety.

The UML diagram below illustrates this structure.

Design choice: Creating ShippingPackage as an interface allows for extensibility, enabling the system to support various package types, like fragile or perishable without modifying core logic.

Account

The Account class represents a customer using the locker system. It maintains the customer's balance for locker rental charges and is associated with their notification preferences, which defines terms for free usage hours and overcharge rates. This design ensures that charges are applied in accordance with each customer's agreement.

This class provides functionality to add funds to the account.

Design choice: We separate AccountLockerPolicy into a separate class to support flexible billing rules based on customer policies. This separation of customer and policy data enhances maintainability and allows for personalized locker usage terms.

AccountLockerPolicy

This class defines the rules and policies for locker usage associated with an account. It includes the number of days during which locker usage is free (freeHoursDuration) and the maximum number of days a package can remain in the locker before it must be cleared (maximumHoldDays). These rules ensure fair usage and prevent long-term storage issues.

NotificationInterface

Defines the contract for sending notifications. It includes a single method, sendNotification(), which takes a message and an account as parameters.

Implementations of this interface range from simple notifications to complex alerts for package pickup availability and locker usage fees.

Best practices: In Date-based systems, separate updates into notifications and other implemented as interfaces to keep the design flexible and scalable while avoiding unnecessary complexity.

LockerManager

The LockerManager class is responsible for managing package storage and retrieval at a specific site, ensuring that packages are assigned to suitable lockers based on size availability. It works with the following components:

- **Map<Integer, Locker> availableLockers**: Tracks available lockers by their IDs.
- **Map<Integer, Locker> occupiedLockers**: Keeps track of lockers that have packages assigned to a locker and its corresponding package.
- **Map<String, Locker> accessCodeMap**: Maps access codes to lockers for managing locker usage and retrieval.
- **Map<String, Locker> accessCodeMap**: Maintains a mapping of access codes to lockers, allowing for quick retrieval during package pick-up.

Design choice: The LockerManager class is designed as a facade to simplify interactions between core objects, providing a single point of control for package and locker management.

Complete Class Diagram

Below is the complete class diagram for the shipping locker service.

Class Diagram of Locker Service

With the class diagram on our party, let's quickly outline the through-code.

Code - Shipping Locker Service

The locker system follows a layered design. The interface layer acts as an implement key components of the shipping locker system in the system, we'll implement several core classes that handle locker operations, package management, and notifications.

Locker and LockerSize

The Locker class represents an individual locker and. Each locker has a fixed size (an instance of LockerSize) and maintains details about its occupancy, access code, and last usage. The LockerSize enum defines standard locker sizes (Small, Medium, Large).

Key functionalities include:

- Assigning a package to a locker, resulting the assignment date, and generating a secure access code.
- Calculating storage charges based on the number of days the package has been stored and the locker's daily rate, according to the system's policy.
- Once the package is picked up, make the locker available for future use.

Below is the implementation of this class:

```
public class Locker {
    private LockerSize size;
    private boolean isOccupied;
    private LocalDateTime lastPickedUpDate;
    // Date when the current package was assigned
    private String accessCode;
    // Access code for retrieving the package
    private double dailyRate = 5.0;

    public Locker(LockerSize size) {
        this.size = size;
    }

    // Assigns a package to this locker and generates an access code
    public void assignPackage(ShippingPackage pkg, Date date) {
        this.isOccupied = true;
        this.lastPickedUpDate = date;
        this.accessCode = generateAccessCode();
    }

    // Retrieves the locker by removing the current package and its details.
    public void clearLocker() {
        this.isOccupied = false;
        this.lastPickedUpDate = null;
        this.accessCode = null;
    }

    // Calculates storage fees based on usage duration and policy
    public double calculateFee(int freeHoursDuration) {
        if (!isOccupied || lastPickedUpDate == null) {
            return 0.0;
        }

        // Calculate hours stored.
        long hoursStored = java.time.Duration.between(lastPickedUpDate,
            java.time.LocalDateTime.now()).toHours();

        // Check if exceeds free period.
        if (hoursStored <= freeHoursDuration) {
            return 0.0;
        }

        // Calculate billable hours.
        int billableHours = (int) (hoursStored - freeHoursDuration);
        double fee = billableHours * dailyRate;

        System.out.printf("Locker %d: %d hours stored, %d free hours, %d billable hours, fee: $%.2f",
            getId(), hoursStored, freeHoursDuration, billableHours, fee);

        return fee;
    }

    // Generates a random access code.
    private String generateAccessCode() {
        return "ACCESS" + (int) (Math.random() * 10000);
    }

    // Getters.
    public int getId() {
        return id;
    }

    public LockerSize getSize() {
        return size;
    }

    public boolean isOccupied() {
        return isOccupied;
    }

    public LocalDateTime getLastPickedUpDate() {
        return lastPickedUpDate;
    }

    public String getAccessCode() {
        return accessCode;
    }

    public double getDailyRate() {
        return dailyRate;
    }
}
```

The LockerSize enum defines different locker sizes, each with an associated daily charge rate and physical dimensions (width, height, and depth). This structure allows the system to manage different locker types and ensure accurate pricing and space allocation based on package requirements.

```
public enum LockerSize {
    // Small locker with 20x20x20 dimensions and $3 daily charge.
    SMALL(
        "Small",
        20.0/12.0*12.0*12.0,
        20.0/12.0*12.0*12.0,
        20.0/12.0*12.0*12.0,
        3.0
    ),
    // Medium locker with 30x30x30 dimensions and $5 daily charge.
    MEDIUM(
        "Medium",
        30.0/12.0*12.0*12.0,
        30.0/12.0*12.0*12.0,
        30.0/12.0*12.0*12.0,
        5.0
    ),
    // Large locker with 40x40x40 dimensions and $10 daily charge.
    LARGE(
        "Large",
        40.0/12.0*12.0*12.0,
        40.0/12.0*12.0*12.0,
        40.0/12.0*12.0*12.0,
        10.0
    );

    // Name of the locker size.
    private String displayName;
    // Maximum height in inches.
    private double maxHeight;
    // Width of the locker in inches.
    private double maxWidth;
    // Depth of the locker in inches.
    private double maxDepth;
    // Daily rate for this locker size.
    private double dailyRate;

    // Constructor.
    private LockerSize(
        String displayName,
        double maxHeight,
        double maxWidth,
        double maxDepth,
        double dailyRate
    ) {
        this.displayName = displayName;
        this.maxHeight = maxHeight;
        this.maxWidth = maxWidth;
        this.maxDepth = maxDepth;
        this.dailyRate = dailyRate;
    }

    // Getters.
    public double getMaxHeight() {
        return maxHeight;
    }

    public double getMaxWidth() {
        return maxWidth;
    }

    public double getMaxDepth() {
        return maxDepth;
    }

    public double getDailyRate() {
        return dailyRate;
    }
}
```

Implementation choice: You might have noticed that we have defined the dimensions for small,large and dimensions in the constructor, since to ensure precision in financial calculations, rather than in double, which could introduce rounding errors.

Site

The Site class represents a physical location containing multiple lockers, organized by size for efficient storage management. It maintains a collection of lockers grouped by size, making locating of the same size as AccountLockerPolicy. This layout ensures quick access to a given size and size-specific operations, simplifying locker assignment.

To efficiently manage lockers, the Site class uses a map-based structure, where:

- Each key is a LockerSize, representing a specific locker category ("Small", "Medium", "Large").
- Each value is a Set containing all lockers of that size.

This design allows the system to quickly find available locker and assign packages accordingly.

Here is the implementation of this class:

```
public class Site {
    // Map of locker size to set of lockers.
    private Map<LockerSize, Set<Locker>> lockers = new HashMap<>();

    // Creates a new site with specified number of lockers for each size.
    public Site(int smallCount, int mediumCount, int largeCount) {
        for (LockerSize size : LockerSize.values()) {
            int count = (size == SMALL) ? smallCount
                        : (size == MEDIUM) ? mediumCount
                        : largeCount;
            lockers.put(size, new HashSet<>());

            // Create and add lockers of the specified size.
            for (int i = 1; i <= count; i++) {
                int lockerId = size.ordinal() * 100 + i;
                Locker locker = new Locker(lockerId, size);
                lockers.get(size).add(locker);
            }
        }
    }

    // Finds an available locker of the specified size.
    public Locker findAvailableLockerForSize(LockerSize size) {
        for (Locker locker : lockers.get(size)) {
            if (!locker.isOccupied()) {
                return locker;
            }
        }
        return null;
    }

    // Checks a package to see if it can fit in an available locker.
    // Returns true if suitable locker is available.
    public boolean canAccommodate(ShippingPackage pkg, Date date) {
        LockerSize requiredSize = pkg.getRequiredLockerSize();
        Locker availableLocker = findAvailableLockerForSize(requiredSize);
        return availableLocker != null && availableLocker.canFitPackage(pkg);
    }

    // Getter.
    public Map<LockerSize, Set<Locker>> getLockers() {
        return lockers;
    }
}
```

Implementation choice: We use a HashSet to store lockers in each size category, ensuring efficient management of unique lockers and quick lookup operations.

BasicShippingPackage

The StandardPackage class represents a standard package within the locker system, implementing the ShippingPackage interface. It encapsulates essential package details such as ID, size, and storage status. Additional attributes (width, height, depth) provide more detailed package specifications.

Here is the implementation of this class:

```
public class BasicShippingPackage implements ShippingPackage {
    // Unique identifier for the package.
    private String uniqueId;
    // Volume of the package.
    private double volume;
    // Indicates if the package is prepared.
    private boolean isPrepared;
    // Indicates if the package is stored.
    private boolean isStored;
    // Indicates if the package is retrieved.
    private boolean isRetrieved;
    // Date when the package was stored.
    private LocalDateTime storedDate;
    // Date when the package was retrieved.
    private LocalDateTime retrievedDate;

    // Creates a new shipping package with specified dimensions.
    public BasicShippingPackage(
        String uniqueId, double volume, double height, double width, double depth,
        boolean isPrepared,
        boolean isStored,
        boolean isRetrieved,
        LocalDateTime storedDate,
        LocalDateTime retrievedDate
    ) {
        this.uniqueId = uniqueId;
        this.volume = volume;
        this.isPrepared = isPrepared;
        this.isStored = isStored;
        this.isRetrieved = isRetrieved;
        this.storedDate = storedDate;
        this.retrievedDate = retrievedDate;
    }

    // Marks the package as prepared.
    public void markAsPrepared() {
        this.isPrepared = true;
    }

    // Updates the package status.
    public void markAsStored() {
        this.isStored = true;
        this.storedDate = LocalDateTime.now();
    }

    // Marks the package as retrieved.
    public void markAsRetrieved() {
        this.isRetrieved = true;
        this.retrievedDate = LocalDateTime.now();
    }

    // Checks if the package is prepared.
    public boolean isPrepared() {
        return isPrepared;
    }

    // Checks if the package is stored.
    public boolean isStored() {
        return isStored;
    }

    // Checks if the package is retrieved.
    public boolean isRetrieved() {
        return isRetrieved;
    }

    // Gets the package volume.
    public double getVolume() {
        return volume;
    }

    // Gets the unique identifier.
    public String getUniqueId() {
        return uniqueId;
    }

    // Gets the storage date.
    public LocalDateTime getStoredDate() {
        return storedDate;
    }

    // Gets the retrieval date.
    public LocalDateTime getRetrievedDate() {
        return retrievedDate;
    }
}
```

A key functionality of this class is determining the appropriate locker size for a package. The getLockerSize() method analyzes all three dimensions (height, width, depth) against predefined locker size limits to ensure proper matching.

Additionally, the updateStorageStatus() method allows the locker system to communicate status updates (e.g., when a package is stored or retrieved) to the user.

Account and AccountLockerPolicy

The Account class represents a user's account within the locker system. It stores basic account details, including the accounts unique identifier and current balance. The associated AccountLockerPolicy defines the customer's usage terms, including free hours and overcharge rates.

The account usageFreeHours is used to track locker usage fees, and funds can be added dynamically using the addFunds(amount) method.

The AccountLockerPolicy manages a user's account within the locker system. It stores basic account details, including the accounts unique identifier and current balance. The associated AccountLockerPolicy defines the customer's usage terms, including free hours and overcharge rates.

The account usageFreeHours is used to track locker usage fees, and funds can be added dynamically using the addFunds(amount) method.

The AccountLockerPolicy manages a user's account within the locker system. It stores basic account details, including the accounts unique identifier and current balance. The associated AccountLockerPolicy defines the customer's usage terms, including free hours and overcharge rates.

The account usageFreeHours is used to track locker usage fees, and funds can be added dynamically using the addFunds(amount) method.

The primary responsibilities of this class include:

- freeHoursDuration: The number of days a user can stay in a locker before further action is required.
- maximumHoldDays: The maximum number of days a package can remain in a locker before further action is required, e.g., notification or penalties.

This policy ensures consistent locker usage terms, ensuring that different users can have varying storage allowances based on their agreements.

```
public class AccountLockerPolicy {
    // Number of days of free storage.
    private int freeHoursDuration;
```

```
    // Maximum number of days can be stored
    final int maximumPeriodDays;

    // Creates a new locker policy with specified free and maximum periods
    public AccountLockerPolicy(int freePeriodDays, int maximumPeriodDays) {
        this.freePeriodDays = freePeriodDays;
        this.maximumPeriodDays = maximumPeriodDays;
    }
    // getter methods are omitted for brevity
}
```

LockerManager

The LockerManager class acts as a facade for the overall locker system. It serves as the central coordinator for the locker system, coordinating package assignments to lockers, package pick-ups, and locker releases. It interacts with Site, which oversees the locker inventory and handles the actual placement of packages in available lockers. The NotificationInterface is responsible for sending user notifications regarding package assignments and pickups.

Additionally, it maintains a Map<String, Account> to store customer accounts, which are used to track locker usage charges. To ensure secure and efficient package retrieval, it uses a Map<String, Locker> to map access codes to their corresponding lockers.

Below is the implementation of this class.

```
public class LockerManager {
    // The site being managed
    private final Site site;
    // Service for sending notifications
    private final NotificationInterface notificationService;
    // Map of account IDs to account objects
    private final Map<String, Account> accounts;
    // Map of access codes to lockers
    private final Map<String, Locker> accessCodeMap = new HashMap<>();

    // Creates a new locker manager for a site
    public LockerManager(
            Site site, Map<String, Account> accounts, NotificationInterface notificationService) {
        this.site = site;
        this.accounts = accounts;
        this.notificationService = notificationService;
    }

    // Assigns a package to an available locker
    public Locker assignPackage(ShippingPackage pkg, Date date) {
        Locker locker = site.placePackage(pkg, date);
        if (locker != null) {
            accessCodeMap.put(locker.getAccessCode(), locker);
            notificationService.sendNotification(
                    "Package assigned to locker" + locker.getAccessCode(), pkg.getUser());
        }
        return locker;
    }

    // Processes package pickup using an access code
    public Locker pickUpPackage(String accessCode) {
        Locker locker = accessCodeMap.get(accessCode);
        if (locker != null && locker.checkAccessCode(accessCode)) {
            try {
                BigDecimal charge = locker.calculateStorageCharges();
                ShippingPackage pkg = locker.getPackage();
                locker.releaseLocker();
                pkg.getUser().addUsageCharge(charge);
                pkg.updateShippingStatus(ShippingStatus.RETRIEVED);
                return locker;
            } catch (MaximumStoragePeriodExceededException e) {
                locker.releaseLocker();
                return locker;
            }
        }
        return null;
    }
    // getter methods are omitted for brevity
}
```

- assignPackage(ShippingPackage pkg, Date date): Assigns a package to an available locker via site.placePackage().
- pickUpPackage(String accessCode): Retrieves the package if the provided access code matches the stored locker. It then:
  - Calculates charges based on locker usage duration.
  - Updates the user's usage charges.
  - Releases the locker and marks the package as retrieved.

> **Best practices:** In complex systems, a manager class like LockerManager is commonly used to centralize operations such as package assignment, retrieval, and notifications. However, to maintain a thin manager class, it is essential to delegate low-level operations, such as finding available lockers, storing packages, and enforcing locker policies, to specialized classes like Site and Locker.

Deep Dive Topic

Now that the basic design is complete, the interviewer might ask you some deep dive questions. Let's check out some of these.

Designing an extensible locker creation system

Currently, lockers are created directly based on predefined sizes, which makes the system rigid and difficult to extend. For example, if we need to introduce a new locker size (e.g., XLARGE) or a specialized locker type (e.g., temperature-controlled lockers), we would have to modify multiple parts of the codebase where lockers are instantiated manually. This increases maintenance overhead and reduces flexibility.

To address this, we can use the Factory design pattern to centralize the creation logic for different types of lockers, making the system more modular and extensible.

Note: To learn more about the Factory Pattern and its common use cases, refer to the **Further Reading** section at the end of this chapter.

Here is the code for the LockerFactory class.

```
class LockerFactory {
    // Creates a new locker of the specified size
    public static Locker createLocker(LockerSize size) {
        return switch (size) {
            case SMALL -> new Locker(LockerSize.SMALL);
            case MEDIUM -> new Locker(LockerSize.MEDIUM);
            case LARGE -> new Locker(LockerSize.LARGE);
            case XLARGE -> new Locker(LockerSize.XLARGE);
        };
    }
}
```

Here's how a LockerFactory class enhances the locker system:

- **Centralized object creation:** The LockerFactory encapsulates the locker instantiation process, so any changes to locker creation (e.g., adding new sizes or types) are localized to this class.
- **Extensibility:** If we need to add new locker sizes or types in the future, the factory class can be easily updated, without modifying core business logic in other parts of the system.
- **Improved readability and maintainability:** Using a factory method to create lockers allows us to separate locker creation from locker usage logic, aligning with the Single Responsibility Principle.

Decoupling event handling with an event-driven approach

Currently, the LockerManager directly invokes NotificationInterface.sendNotification() to notify customers when key events occur, such as package assignment or locker expiration. While this approach works, it tightly couples LockerManager with the notification system, meaning any changes to how notifications are sent require modifying LockerManager itself.

How can we improve the system design to make event handling more flexible and scalable?

To decouple event handling from LockerManager, we can use an event-driven approach where multiple components can subscribe to and respond to system events dynamically. Instead of directly calling sendNotification(), LockerManager will broadcast events to registered observers, ensuring a more modular and extensible system.

By implementing the Observer pattern, we can send notifications to customers or administrative staff when certain events occur, such as when a package is delivered or a locker exceeds its maximum usage period.

Note: To learn more about the Observer Pattern and its common use cases, refer to the **Elevator System** chapter.

Instead of hardcoding the notification logic in the LockerManager, the locker system can maintain a list of observers. These observers can be various notification services (e.g., email, SMS) or other systems (e.g., analytics, metrics). When a key event occurs, the LockerManager can notify all registered observers, allowing the system to be easily extended without modifying the core logic.

Below is the implementation of the relevant interface and classes for the observer pattern.

- **Subject**: LockerManagerChange, which maintains a list of observers and notifies them of state changes, such as package assignments.

```
class LockerManagerChange {
    // List of observers that will be notified of locker events
    private final List<LockerEventObserver> observers = new ArrayList<>();

    public void addObserver(LockerEventObserver observer) {
        observers.add(observer);
    }

    public void removeObserver(LockerEventObserver observer) {
        observers.remove(observer);
    }

    private void notifyObservers(String message, Account account) {
        for (LockerEventObserver observer : observers) {
            observer.update(message, account);
        }
    }

    // Assigns a package to an available locker and notifies observers.
    public void assignPackage(ShippingPackage pkg) {
        Locker locker = assignLockerToPackage(pkg);
        if (locker != null) {
            notifyObservers("Package assigned to locker", pkg.getUser());
        }
    }

    private Locker assignLockerToPackage(ShippingPackage pkg) {
        return null;
    }
}
```

- **Observer**: The LockerEventObserver interface, which defines the update() method that observers must implement to handle notifications.

```
// Interface for objects that need to be notified of locker events
public interface LockerEventObserver {
    // Updates the observer with a message and the affected account
    void update(String message, Account account);
}
```

- **Concrete Observers**: Classes like EmailNotification, which implement LockerEventObserver to send email alerts for specific events, enabling flexible and extensible event handling.

```
// Implementation of LockerEventObserver that sends email notifications
class EmailNotification implements LockerEventObserver {
    // Updates the observer with a message and sends it to the account owner
    public void update(String message, Account account) {
        // send email to account owner
    }
}
```

Wrap Up

In this chapter, we designed a Shipping Locker System. We began by clarifying requirements through a structured Q&A discussion, similar to an interviewer-candidate exchange. From there, we identified core objects and developed a class diagram to represent the system's structure. We implemented the key components of the locker service, bringing the design to life.

In the deep dive section, we explored advanced topics, including how the Factory Pattern simplifies locker instantiation for future scalability and how the Observer Pattern decouples event handling, enabling flexible notifications.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Further Reading: Factory Design Pattern

This section gives a quick overview of the design patterns used in this chapter. It's helpful if you're new to these patterns or need a refresher to better understand the design choices.

Factory design pattern

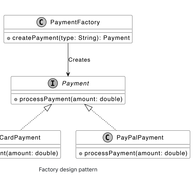
The Factory Method Pattern is a creational design pattern that provides an interface for creating instances of a class but allows subclasses to alter the type of objects that will be created.

In the shipping locker system, we use the Factory pattern through the LockerFactory class to centralize locker creation, enabling easy addition of new locker sizes (e.g., XLARGE) without modifying existing code. To illustrate the Factory pattern in another domain, the following example uses an e-commerce payment system.

Problem

Imagine building an e-commerce payment system that initially supports only credit card payments, with logic tied to a CreditCardPayment class. As demand grows for new payment options like digital wallets or cryptocurrency, adding them requires altering multiple parts of the tightly coupled codebase, increasing maintenance effort, and reducing flexibility.

Solution



Factory design pattern

The Factory Method pattern addresses this by replacing direct object creation (using the new operator) with a factory method that encapsulates instantiation. This allows new payment types to be added via subclasses without changing existing code, decoupling payment processing from the main system for better scalability and maintainability.

When to use

The factory design pattern is particularly useful in the following scenarios:

- Use the Factory Method when the specific types and dependencies of objects required by your code are unknown in advance.
- Use the Factory Method to allow users of your library or framework to extend its internal components without modifying existing code.
- Use the Factory Method to optimize resource utilization by reusing existing objects rather than creating new instances each time.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor
Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information
Support
hi@bytebytego.com
Report a Bug
Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews
Ask Alex
Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.